

Lecture: Unsupervised Learning (Clustering)

Author: Refat Othman

0) Lecture Overview

Unsupervised Learning = Learning without a teacher

- You only have **X (features)**.
- You do **NOT** have **y (labels)**.
- Goal: the model discovers **structure** in data by itself.

Typical outputs

- Groups/segments (Clustering)
- Unusual points (Anomaly Detection)

Real example

- You have users' browsing behavior (time on site, pages visited, categories, clicks).
- There is no column like "User Type".
- Unsupervised learning helps you discover *possible* segments.

1) Learning Objectives

1. Explain what "unsupervised" means and when to use it
2. Cluster data using **K-Means**
3. Choose a reasonable **k** using **Elbow** and **Silhouette**
4. Interpret clusters using feature summaries

2) Key Concepts

2.1 Supervised vs Unsupervised (Core idea)

- **Supervised**: learn mapping $X \rightarrow y$ because you have labels
- **Unsupervised**: only X , so you discover patterns (groups/structure)

Important: Unsupervised results are *hypotheses* (not guaranteed truth). We validate with metrics + domain reasoning.

2.2 Similarity & Distance (why it matters)

Most clustering depends on "distance" between points. Common distance: **Euclidean distance**.

If one feature has larger numeric scale, it dominates the distance. So we usually need **feature scaling**.

2.3 Feature Scaling (VERY important for clustering)

Why do we scale?

K-Means (and many clustering methods) use **distance** to decide which points are “close”. If one feature has much larger numbers than the others, it **dominates the distance**. That means the algorithm will mostly cluster based on that large-scale feature — even if it’s not the most important one.

Example (simple numbers):

- Feature 1: **Income** (0–200000)
- Feature 2: **Spending score** (0–100)

Suppose we compare two users:

- User A: income = 50,000, spending = 90
- User B: income = 60,000, spending = 10

Without scaling (rough idea):

- Income difference = 10,000 (very large)
- Spending difference = 80 (small compared to 10,000) So the distance is dominated by income, and the model mostly ignores spending.

After scaling: Income and spending become comparable (both in “standard deviation units”), so **both features influence the clustering**.

What does StandardScaler do?

StandardScaler converts each feature column to:

- mean = 0
- std = 1

Formula (per feature column):

- $x_{\text{scaled}} = (x - \text{mean}) / \text{std}$

3) Unsupervised Learning Type #1: Clustering

3.1 What is clustering?

Clustering = grouping points that are similar. We don’t know group names in advance. We just find structure.

Use cases

- Customer segmentation
- Grouping similar documents
- Grouping images by visual similarity
- Exploration of a new dataset

4) Algorithm 1: K-Means Clustering

4.1 Intuition

K-Means tries to find **k centers** ("centroids"). Each point belongs to the closest centroid.

4.2 What K-Means is optimizing

K-Means tries to minimize:

- the sum of squared distances from each point to its cluster center

4.3 The algorithm steps (easy to teach)

1. Choose **k** initial centers (randomly or smart init)
2. Assign each point to the closest center
3. Recompute centers as the mean of assigned points
4. Repeat until assignments stop changing

4.4 Important assumptions and limitations

K-Means works best when:

- clusters are roughly **spherical** (round)
- clusters have similar density/size
- distance in feature space is meaningful

K-Means struggles when:

- clusters are non-spherical (moon shapes)
- data has strong outliers
- clusters vary a lot in density

5) LAB 1 — K-Means using Built-in Scikit-Learn Datasets (Beginner-Friendly)

5.1 Setup

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
```

Line-by-line interpretation

- `import numpy as np`
 - Imports **NumPy** for numerical work (arrays, math). We may use it later (e.g., counts, reshaping).
- `import matplotlib.pyplot as plt`

- Imports plotting library **matplotlib** so we can visualize the data and the clusters.
 - `from sklearn.datasets import make_blobs`
 - Imports a helper that **generates fake clustered data** (like “students points” already coming from different groups).
 - This is great for learning because we can see what clustering should do.
 - `from sklearn.cluster import KMeans`
 - Imports the **K-Means** algorithm class.
-

5.2 Create data (pretend we have no labels)

```
X, _ = make_blobs(
    n_samples=400,      # total number of data points to generate
    centers=4,           # how many true groups (clusters) exist in the generated
    data                # data
    cluster_std=1.2,     # how spread out each group is (higher = more overlap)
    random_state=42      # makes results reproducible (same data every run)
)

plt.scatter(X[:, 0], X[:, 1])
plt.title("Unlabeled data (students only see X)")
plt.show()
```

Line-by-line interpretation

- `X, _ = make_blobs(...)`
 - `make_blobs` returns **two things**:
 1. `X`: the feature matrix (shape: `n_samples x n_features`) → this is what the model sees
 2. `y`: the true cluster label used to generate the data → we **ignore it** in unsupervised learning
 - `_` means: “I’m receiving a value but I won’t use it.”
- `n_samples=400`
 - Creates 400 points (more points = clearer visualization).
- `centers=4`
 - The generator makes 4 real groups.
 - **Important:** In real life we don’t know this. Here we use it for learning.
- `cluster_std=1.2`
 - Controls how “wide” each blob is.
 - Bigger value → more overlap → clustering becomes harder.

- `random_state=42`
 - Fixes randomness so every student gets the same dataset.
- `plt.scatter(X[:, 0], X[:, 1])`
 - Plots a scatter chart.
 - `X[:, 0]` = first feature column, `X[:, 1]` = second feature column.
- `plt.title(...)`
 - Adds a title to the plot.
- `plt.show()`
 - Displays the plot.

Teaching tip: Ask students: "Can you visually guess how many groups you see?"

5.3 Train K-Means

```
k = 4
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=clusters)
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker="X", s=200
)
plt.title(f"K-Means Clustering (k={k})")
plt.show()
```

Line-by-line interpretation

- `k = 4`
 - We choose the number of clusters we want K-Means to find.
 - In real datasets, we usually test multiple values of `k`.
- `kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)`
 - Creates a K-Means model object.
 - `n_clusters=k` tells it how many clusters to create.
 - `random_state=42` ensures consistent initialization across runs.
 - `n_init=10` means K-Means will run **10 times** with different initial centers and keep the best result (more stable).
- `clusters = kmeans.fit_predict(X)`

- **Two actions in one line:**
 - `fit(X)` learns cluster centers from the data.
 - `predict(X)` assigns each point to a cluster.
- Output: `clusters` is an array of length 400 (one cluster label per point), values like 0,1,2,2.
- `plt.scatter(X[:, 0], X[:, 1], c=clusters)`
 - Plots points again, but now uses `c=clusters` to color each point by its assigned cluster.
- `plt.scatter(kmeans.cluster_centers_[ :, 0], kmeans.cluster_centers_[ :, 1], marker="X", s=200)`
 - Plots the **centroids** (cluster centers).
 - `cluster_centers_` contains the coordinates of the centers.
 - `marker="X"` makes centers visible.
 - `s=200` makes the marker bigger.
- `plt.title(f"K-Means Clustering (k={k})")`
 - Title with the chosen k.
- `plt.show()`
 - Shows the final clustered plot.

Mini-check questions:

1. What happens if we change `k` to 2 or 6?

5.4 In-class exercise (5–10 min)

Task

- Change `k` to 2, 3, 5
- Observe what happens
- Answer:
 - What happens when k is too small?
 - What happens when k is too large?

6) Choosing k (How many clusters should we use?)

Big idea: In real data, we **don't know** the true number of groups. So we try different values of **k** and use **metrics + judgment**.

6.1 Method A: Elbow Method (Inertia)

What is inertia?

- Inertia = *sum of squared distances* from each point to its assigned cluster center.
- Smaller inertia means points are closer to their centers.

Why does elbow help?

- When k increases, inertia **always decreases** (because more clusters = easier to fit).
- We look for the point where adding more clusters gives only **small improvement**.

```
inertias = []
K = range(1, 11)

for k in K:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    inertias.append(km.inertia_)

plt.plot(list(K), inertias, marker="o")
plt.xlabel("k")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```

Line-by-line interpretation (Elbow code)

- `inertias = []`
 - Create an empty list to store inertia values (one value for each k).
- `K = range(1, 11)`
 - We will test k from 1 to 10.
- `for k in K:`
 - Loop over each possible k.
- `km = KMeans(n_clusters=k, random_state=42, n_init=10)`
 - Create a K-Means model with the current k.
 - `random_state=42` for reproducibility.
 - `n_init=10` runs K-Means 10 times with different starting centers and keeps the best.
- `km.fit(X)`
 - Train K-Means on the dataset `X` (learn the centers for this k).
- `inertias.append(km.inertia_)`
 - After fitting, `km.inertia_` stores the inertia value.
 - Append it to the list so we can plot inertia vs k.
- `plt.plot(list(K), inertias, marker="o")`

- Plot k values on the x-axis and inertia on the y-axis.
- `marker="o"` makes each point visible.
- `plt.xlabel("k"), plt.ylabel("Inertia")`
 - Label axes so the plot is readable.
- `plt.title("Elbow Method")`
 - Add a plot title.
- `plt.show()`
 - Display the elbow plot.

How students interpret the elbow plot

- Look for a “bend” (elbow): the k after which the line flattens.
- That k is often a reasonable choice.

Teaching note: Elbow is visual and subjective. Use it with silhouette + interpretation.

6.2 Method B: Silhouette Score (Often better)

What does silhouette measure? It compares:

- **a** = average distance to points in the same cluster (cohesion)
- **b** = average distance to points in the nearest other cluster (separation)

Silhouette per point is roughly: $(b - a) / \max(a, b)$

Range: **-1 to +1**

- close to **+1** → well separated clusters
- around **0** → overlapping clusters
- negative → many points might be assigned to the wrong cluster

```
from sklearn.metrics import silhouette_score

scores = []
K = range(2, 11)

for k in K:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X)
    scores.append(silhouette_score(X, labels))

plt.plot(list(K), scores, marker="o")
plt.xlabel("k")
plt.ylabel("Silhouette Score")
plt.title("Silhouette vs k")
plt.show()
```


Line-by-line interpretation (Silhouette code)

- `from sklearn.metrics import silhouette_score`
 - Import the function that computes silhouette score.
- `scores = []`
 - Create an empty list to store silhouette scores.
- `K = range(2, 11)`
 - Start from k=2 because silhouette score is not meaningful for k=1 (only one cluster).
- `for k in K:`
 - Loop through each k to test.
- `km = KMeans(n_clusters=k, random_state=42, n_init=10)`
 - Create K-Means model for this k.
- `labels = km.fit_predict(X)`
 - Fit K-Means and assign each point to a cluster.
 - `labels` is an array like `[0, 0, 1, 2, ...]`.
- `scores.append(silhouette_score(X, labels))`
 - Compute silhouette score using the original data `X` and the cluster labels.
 - Append it to the `scores` list.
- `plt.plot(list(K), scores, marker="o")`
 - Plot k on x-axis, silhouette score on y-axis.
- `plt.xlabel("k"), plt.ylabel("Silhouette Score")`
 - Label axes.
- `plt.title("Silhouette vs k")`
 - Title.
- `plt.show()`
 - Show the plot.

How students interpret the silhouette plot

- Choose the k with the **highest silhouette score** (often a good default).
- If there are multiple close values, pick the **simpler** k (smaller k) and justify with interpretation.

7) LAB 2 — Real Dataset Example (See the Data → Then Cluster It)

Example: Iris Dataset (Built-in in Scikit-Learn)

The Iris dataset contains measurements of flowers:

- sepal length
- sepal width
- petal length
- petal width

There are 150 samples.

Even though the dataset has true labels (3 species), we will **pretend we don't know them** and use it as an unsupervised problem.

Step 1: Load and Explore the Dataset (Before Clustering)

Students must first *see the data structure* before applying K-Means.

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X = iris.data # we pretend y does not exist

# Scale for K-Means
X_scaled = StandardScaler().fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

print("Cluster counts:", np.bincount(clusters))
```

8.1 Interpret clusters using feature averages

```
import pandas as pd

df = pd.DataFrame(X, columns=iris.feature_names)
df["cluster"] = clusters

print(df.groupby("cluster").mean())
```

In-class exercise (interpretation)

- Which cluster has the largest petal length on average?
- Which cluster seems most distinct?

9) Assignments (homework)

Assignment: Customer Segmentation (CSV)

Dataset idea: Mall customers / e-commerce user metrics (any CSV with numeric columns)

Dataset link: <https://www.kaggle.com/datasets/shwetabh123/mall-customers>

Tasks

1. Load dataset with pandas
2. Select numeric features
3. Handle missing values (drop or fill)
4. Scale with StandardScaler
5. Try K-Means with $k = 2..10$
6. Plot elbow + silhouette
7. Choose best k and justify